МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Д. Д. Баженова
Группа: М8О-301Б-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель - загрузить изображения из файловой системы, создать текстуры из изображений и, используя текстурные координаты в вершинах, а также сэмплеры, наложить их на рисуемые объекты.

- Используйте код предыдущей лабораторной работы, чтобы внести поддержку текстур в качестве альбедо (основного цвета) вместо использования сплошного цвета или цвета для вершин.
- Изображения должны быть загружены из файловой системы. Формат файлов растровой графики может быть любым: PNG, JPEG, GIF, BMP, TIFF, TGA и др. Используйте сторонние библиотеки для загрузки и декодирования изображений
- Программа должна работать в реальном времени, к элементам пользовательского интерфейса следует добавить выбор текстуры из выпадающего списка, если их будет несколько.
- У вершин, помимо атрибутов позиции и нормали, должны быть текстурные координаты. Для большей части фигур, текстурные координаты можно сгенерировать процедурно вместе с позицией и нормалями.
- Для одного из объектов на сцене должно быть реализовано нетривиальное сэмплирование (примеры в чате).
- Для контроля использования нетривиального сэмплирования текстуры для объектов следует использовать флаги или параметры в push константах вместо введение новых данных в структуру свойств моделей в uniform буфере.
- Используйте один динамический uniform буфер (массив из структур свойств моделей) вместо множества uniform буферов для каждой модели. Сдвиг чтения структуры со свойствами модели динамического uniform буфера должен задаваться в команде привязки наборов дескрипторов к текущему графическому конвейеру.
- Переведите память для вершинных и индексных данных на GPU вместо RAM, по аналогии с копированием текстур в VRAM. Теперь вершинные и индексные буферы должны иметь память в VRAM. Используйте staging буфер и команды GPU на копирование из staging буфера.

Метод решения

Изображения загружаются из файловой системы с помощью библиотеки stb_image в формате RGBA8. В оперативной памяти (RAM) создается массив пикселей.

Далее создается объект VkImage с форматом VK_FORMAT_R8G8B8A8_SRGB (для альбедо) или UNORM (для служебных карт), выделяется память на GPU. Изображение переводится в состояние VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL с помощью VkImageMemoryBarrier, после чего в него копируются данные из временного буфера. В конце изображение переводится в VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL, чтобы шейдер мог его читать.

Поскольку CPU не имеет прямого доступа к памяти видеокарты (VRAM), используется промежуточный staging буфер, который позволяет прокидывать через себя данные (массивы вершин и индексов), копируя их из RAM в VRAM, после чего этот буфер уничтожается.

Вместо создания отдельного UBO и дескриптора для каждой модели, создается один большой буфер, содержащий массив структур ModelUniforms. Для этого используется тип дескриптора VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC. При привязке дескриптора (в vkCmdBindDescriptorSets) передается динамический сдвиг (dynamicOffset), который указывает GPU, с какого места в массиве читать матрицу текущей модели.

Каждая текстура оборачивается в VkSampler (определяет фильтрацию и адресацию). Для каждого материала создается свой VkDescriptorSet, который объединяет в себе 3 сэмплера: albedo_texture, specular_texture, emissive_texture. Во время рендера нужный набор привязывается перед вызовом отрисовки.

Для передачи параметров, которые часто меняются (флаг включения эффекта и текущее время), используются Push-константы. Это позволяет передавать данные напрямую в конвейер без использования буферов.
В фрагментном шейдере реализован эффект искажения UV-координат и пикселизации, зависящий от времени.

Для нетривиального сэмплирования в шейдере UV-координаты сначала искажаются волной, а затем квантуются (пикселизируются). Вычисляется расстояние от центра, размеры пикселя динамически меняются от времени t, после координаты округляются вниз с помощью функции floor для создания блочного эффекта. Для эффекта пульсации итоговый цвет альбедо модулируется.

Результаты

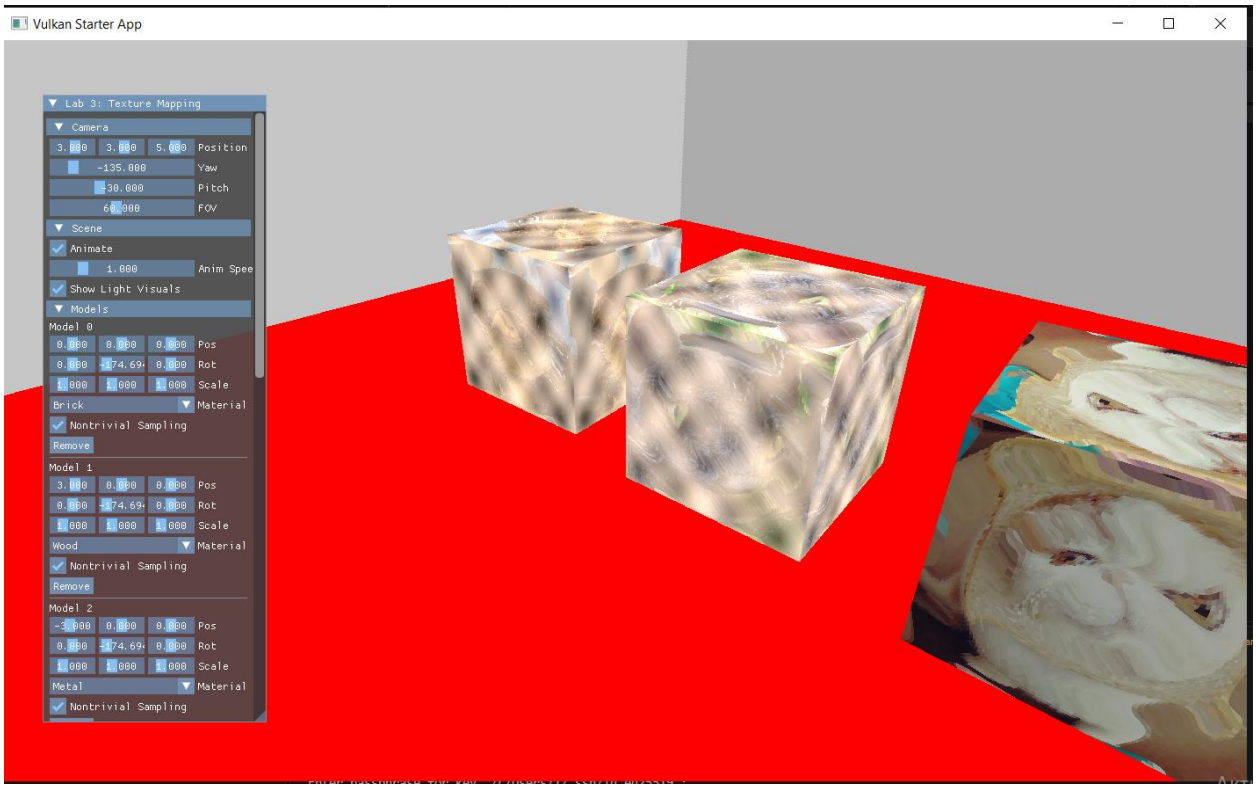


Рисунок 1. Запуск приложения

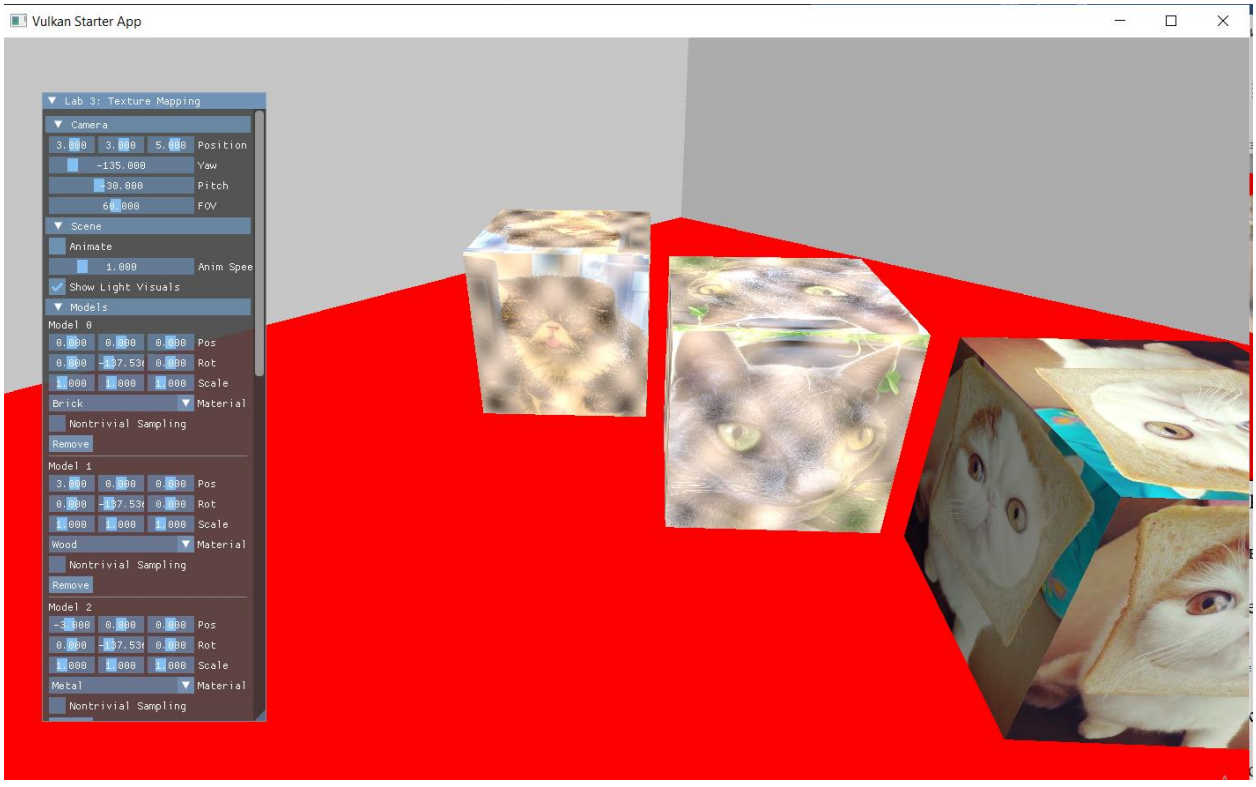


Рисунок 2. Кубы без сэплирования

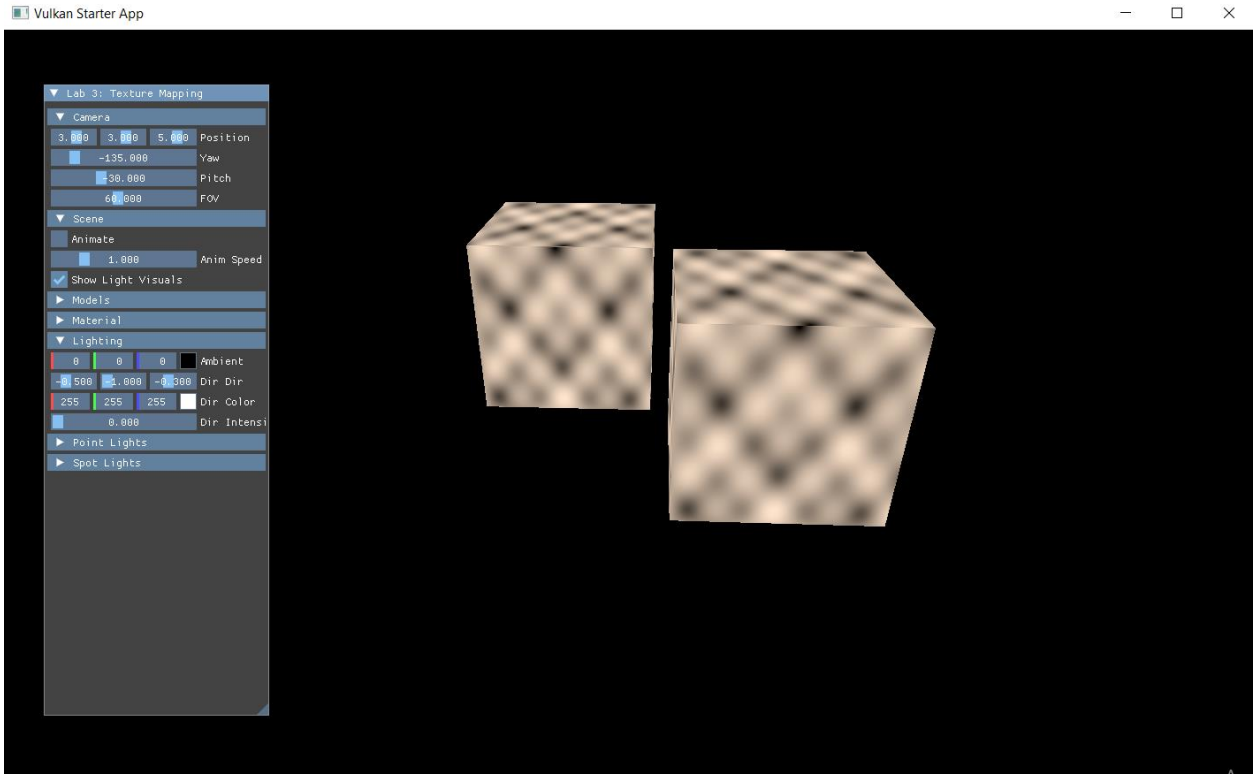


Рисунок 3. Поведение в темноте

Выводы

В ходе лабораторной работы была реализована сцена с моделями, покрытыми текстурами. Были применены динамические UBO для минимизации выделяемых дескрипторов, push-константы для управления шейдерами в процессе смены действий на сцене, а также staging буферы для переноса графики в память GPU. Освоены методы процедурной генерации текстур и алгоритмы искажения координат при сэмплировании.